



Hey, I'm Stefanie

Scikit-learn maintainer

Open Source Developer @probabl

Connecting

 <https://www.linkedin.com/in/StefanieSenger>
 <https://github.com/StefanieSenger>



:probabl.



Scikit-learn's Metadata Routing API



Stefanie Senger



@StefanieSenger



EuroSciPy Kraków
July 20, 2026

:probabl.

What is metadata routing?

Introducing scikit-learn's metadata routing API

Metadata

- Data to apply on top of tabular data
- Data a function can handle besides X and y

Use

- Fairness related, business logic, custom behaviour, etc.
- Benefit won't be a better score, but a more realistic model

Routing

- Passing metadata between several components of a data science pipeline

What is metadata routing?

Introducing scikit-learn's metadata routing API

set_config(enable_metadata_routing=False), the default

- Restricted use of `sample_weight`
- No metadata passing in nested structures

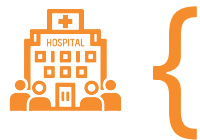
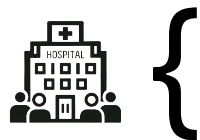
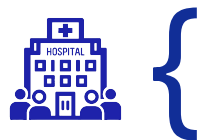
set_config(enable_metadata_routing=True)

- Nested `sample_weight` and `groups` passing
- Using metadata from other libraries' objects together with scikit-learn objects
- Custom definition of new metadata

Passing metadata without the routing API

Data from real world observations
(not from double blind randomised trial)

Target



Sex	Age	Severity	Medication
Male	88	6	1
Female	100	4	0
Female	55	10	1
Female	70	7	1
Male	65	5	0
Male	68	0	1

Recovery Time
33
9
10
25
70
29

Passing metadata without the routing API

In depth into the “groups” metadata in scikit-learn

data				groups	target
Sex	Age	Severity	Medication	Hospital	Recovery Time
Male	88	6	1	Blue	33
Female	100	4	0	Blue	9
Female	55	10	1	Black	10
Female	70	7	1	Black	25
Male	65	5	0	Orange	70
Male	68	0	1	Orange	29

Passing metadata without the routing API

Groups handling during cross-validation using GroupKFold

Training set

Validation set

Fold #1



Fold #2



Fold #3



Passing metadata without the routing API

In depth into the “groups” metadata in scikit-learn

```
from sklearn.model_selection import GroupKFold, cross_validate
from sklearn.linear_model import Ridge
```

```
ridge = Ridge()
```

```
# GroupKFold considers groups while splitting:
```

```
cv = GroupKFold(n_splits=2)
```

```
# providing `groups` for GroupKFold splitter:
```

```
cross_validate(ridge, X, y, cv=cv, groups=groups)
```

```
{'fit_time': array([0.00135875, 0.00036836]),
 'score_time': array([0.00036907, 0.00023532]),
 'test_score': array([-0.2923336 , -0.12131859])}
```

Passing metadata without the routing API

Using the “sample_weight” metadata in scikit-learn

`sample_weight` draws a model’s attention to a sub-group of samples, in order to adjust their distribution (e.g. IPTW in medical settings)

- in distinction to a randomised trial, real-world observations often come with imbalanced and biased data

Sex	Age	Severity	Medication
Male	88	6	1
Female	100	4	0
Female	55	10	1

Recovery Time
33
9
10

Passing metadata without the routing API

Using the “sample_weight” metadata in scikit-learn

```
from sklearn.model_selection import cross_validate
from sklearn.linear_model import Ridge

# Ridge().fit() can consume `sample_weight`
ridge = Ridge()

cross_validate(ridge, X, y, cv=cv, sample_weight=sample_weight)
```

TypeError: got an unexpected keyword argument 'sample_weight'

Passing metadata without the routing API

```
ridge = Ridge() # Ridge().fit() can consume sample_weight
```

```
cv=GroupKFold(n_splits=2) # consumes groups
```

```
scoring = get_scorer("neg_mean_squared_error") # can consume sample_weight
```

```
search = GridSearchCV(  
    ridge, param_grid={"alpha": [0.1, 1, 10]}, cv=cv, scoring=scoring  
)
```

```
cross_validate(  
    search, X, y, cv=cv, scoring=scoring,  
    sample_weight=sample_weight, groups=groups,  
)
```

- Cross-validation on a `GridSearchCV` that searches `Ridge` hyperparameters
- Passing `sample_weight` and `groups` into it

`TypeError: got an unexpected keyword argument 'sample_weight'`

Using the metadata routing API

```
ridge = Ridge()
```

```
cv=GroupKFold(n_splits=2)
```

```
scoring = get_scorer("neg_mean_squared_error")
```

```
search = GridSearchCV(  
    ridge, param_grid={"alpha": [0.1, 1, 10]}, cv=cv, scoring=scoring  
)
```

```
cross_validate(  
    search, X, y, cv=cv, scoring=scoring,  
    params={"sample_weight": sample_weight, "groups": groups},  
)
```

Using the metadata routing API

```
import sklearn
```

```
sklearn.set_config(enable_metadata_routing=True)
```

```
ridge = Ridge()
```

```
cv=GroupKFold(n_splits=2)
```

```
scoring = get_scorer("neg_mean_squared_error")
```

```
search = GridSearchCV(
```

```
    ridge, param_grid={"alpha": [0.1, 1, 10]}, cv=cv, scoring=scoring
```

```
)
```

```
cross_validate(
```

```
    search, X, y, cv=cv, scoring=scoring,
```

```
    params={"sample_weight": sample_weight, "groups": groups},
```

```
)
```

1. Enable metadata routing

Using the metadata routing API

```
import sklearn
```

```
sklearn.set_config(enable_metadata_routing=True)
```

```
ridge = Ridge()
```

```
cv=GroupKFold(n_splits=2)
```

```
scoring = get_scorer("neg_mean_squared_error")
```

```
search = GridSearchCV(
```

```
    ridge, param_grid={"alpha": [0.1, 1, 10]}, cv=cv, scoring=scoring
```

```
)
```

```
cross_validate(
```

```
    search, X, y, cv=cv, scoring=scoring,
```

```
    params={"sample_weight": sample_weight, "groups": groups},
```

```
)
```

1. Enable metadata routing
2. Pass metadata to top level tool

Using the metadata routing API

```
import sklearn
```

```
sklearn.set_config(enable_metadata_routing=True)
```

```
ridge = Ridge().set_fit_request(sample_weight=True)
```

```
cv=GroupKFold(n_splits=2)
```

```
scoring = get_scorer("neg_mean_squared_error")
```

```
scoring.set_score_request(sample_weight=True)
```

```
search = GridSearchCV(
```

```
    ridge, param_grid={"alpha": [0.1, 1, 10]}, cv=cv, scoring=scoring
```

```
)
```

```
cross_validate(
```

```
    search, X, y, cv=cv, scoring=scoring,
```

```
    params={"sample_weight": sample_weight, "groups": groups},
```

```
)
```

1. Enable metadata routing
2. Pass metadata to top level tool
3. Set requests where metadata is consumed

Transforming validation sets in pipelines

- estimators that split off validation sets internally may cause data leakage
- validation sets should always be split off before any transformation
- `transform_input` param in Pipeline defines a set that gets transformed along with `X` before it gets consumed inside the method that requested it
- our own HGBT (HistGradientBoosting) uses a validation set for early stopping; LightGBM compatible with `transform_input`; XGBoost not yet

Transforming validation sets in pipelines

```
sklearn.set_config(enable_metadata_routing=True)

X, y = make_classification(random_state=0)

X_train, X_val, y_train, y_val = train_test_split(X, y,
test_size=0.2, random_state=0)

clf =
HistGradientBoostingClassifier(early_stopping=True).set_fit_request(
    X_val=True, y_val=True
)

pipe = Pipeline(
    [("scaler", StandardScaler()), ("clf", clf)],
    transform_input=["X_val"],
)

pipe.fit(X_train, y_train, X_val=X_val, y_val=y_val)
```

Transforming validation sets in pipelines

```
sklearn.set_config(enable_metadata_routing=True)

X, y = make_classification(random_state=0)

X_train, X_val, y_train, y_val = train_test_split(X, y,
test_size=0.2, random_state=0)

clf =
HistGradientBoostingClassifier(early_stopping=True).set_fit_request(
    X_val=True, y_val=True
)

pipe = Pipeline(
    [("scaler", StandardScaler()), ("clf", clf)],
    transform_input=["X_val"],
)

pipe.fit(X_train, y_train, X_val=X_val, y_val=y_val)
```

We want a validation set used in HGBT to evaluate early stopping.

Transforming validation sets in pipelines

```
sklearn.set_config(enable_metadata_routing=True)

X, y = make_classification(random_state=0)

X_train, X_val, y_train, y_val = train_test_split(X, y,
test_size=0.2, random_state=0)

clf =
HistGradientBoostingClassifier(early_stopping=True).set_fit_request(
    X_val=True, y_val=True
)

pipe = Pipeline(
    [("scaler", StandardScaler()), ("clf", clf)],
    transform_input=["X_val"],
)

pipe.fit(X_train, y_train, X_val=X_val, y_val=y_val)
```

We want a validation set used in HGBT to evaluate early stopping.

Passing ["X_val"] as transform_input transforms it in every step of the Pipeline up until the point where it is consumed in HGB.

Transforming validation sets in pipelines

```
sklearn.set_config(enable_metadata_routing=True)

X, y = make_classification(random_state=0)

X_train, X_val, y_train, y_val = train_test_split(X, y,
test_size=0.2, random_state=0)

clf =
HistGradientBoostingClassifier(early_stopping=True).set_fit_request(
    X_val=True, y_val=True
)

pipe = Pipeline(
    [("scaler", StandardScaler()), ("clf", clf)],
    transform_input=["X_val"],
)

pipe.fit(X_train, y_train, X_val=X_val, y_val=y_val)
```

We want a validation set used in HGBT to evaluate early stopping.

Passing ["X_val"] as transform_input transforms it in every step of the Pipeline up until the point where it is consumed in HGB.

Passing validation set into top level method.

Metadata routing API

Recent updates and ongoing work

- `TargetEncoder` can now use group splitters [#33089](#)
- Metadata usage inside callbacks to come (`ScoringMonitor`) [#34137](#)
- Developer API for metadata routing to come [#34314](#)
- Default requests to come [#31413](#)
- Visualisation and debugging tools to come [#31535](#)
- Metadata Routing still experimental, but pretty mature

Metadata routing API

References and Links

Metadata routing

- [User Guide on Metadata Routing](#)
- [Developer Guide on Metadata Routing](#)
- [Adrin Jalali's talk on the internal logic of metadata routing](#)

Use cases using `sample\_weight`

- [:probabl. Whiteboard Series by Vincent Warmerdam: Improving models via subsets](#)
- [Blogpost by Florian Wilhelm on Inverse Probability of Treatment Weighting](#)

Metadata routing API

Thank you for your attention!

Time for your questions

Connecting

<https://www.linkedin.com/in/StefanieSenger>

<https://github.com/StefanieSenger>